

试炼密室生成逻辑分析报告-1.21.11

Minecraft 1.21.11 试炼密室（Trial Chambers）生成逻辑分析报告

分析对象：官方 1.21.11 源码（Mojang 官方映射，经 Loom + Vineflower 反编译）

关键结论：1.21.11 中 TrialChamberData / TrialChamberGenerator /

TrialChamberPiece 三个类已不存在，试炼密室已完全改为数据驱动（Jigsaw structure pool + JSON 数据 + TrialSpawner 系列运行时类）。下面的分析基于其替代实现。

0. 类映射：1.21.1 → 1.21.11

1.21.1 旧类（已删除）	1.21.11 替代实现
net.minecraft.structure.TrialChamberData	net.minecraft.data.worldgen.Tr (池注册) + data/minecraft/worldgen/templ (JSON 池) + data/minecraft/st (模板 NBT)
net.minecraft.structure.TrialChamberGenerator	data/minecraft/worldgen/structu (JigsawStructure 配置) + data/minecraft/worldgen/structu (网格放置)
net.minecraft.structure.TrialChamberPiece	PoolElementStructurePiece (通用 net.minecraft.world.level.level 归放置器 JigsawPlacement\$Placer

1. 放置规则（34×34 区块网格判定）

1.1 数据源：

data/minecraft/worldgen/structure_set/trial_chambers.json

```
{
  "placement": {
    "type": "minecraft:random_spread",
    "salt": 94251327,
    "separation": 12,
    "spacing": 34
  },
}
```

```
"structures": [{ "structure": "minecraft:trial_chambers", "weight": 1 }]
```

- **spacing = 34**（网格周期，即 34×34 chunk 网格）
- **separation = 12**（相邻密室最小间隔，取值为 22 = 34-12 的随机偏移）
- **salt = 94251327**（放置种子的固定盐值）

1.2 核心算法：

RandomSpreadStructurePlacement.getPotentialStructureChunk()

```
//  
net.minecraft.world.level.levelgen.structure.placement.RandomSpreadStructure  
Placement  
public ChunkPos getPotentialStructureChunk(long seed, int chunkX, int  
chunkZ) {  
    int regionX = Math.floorDiv(chunkX, this.spacing); // spacing = 34  
    int regionZ = Math.floorDiv(chunkZ, this.spacing);  
    WorldgenRandom random = new WorldgenRandom(new LegacyRandomSource(0L));  
    random.setLargeFeatureWithSalt(seed, regionX, regionZ, this.salt()); //  
    salt = 94251327  
    int range = this.spacing - this.separation; // 22  
    int ox = this.spreadType.evaluate(random, range); // LINEAR →  
    nextInt(22) ∈ [0,21]  
    int oz = this.spreadType.evaluate(random, range); // LINEAR →  
    nextInt(22) ∈ [0,21]  
    return new ChunkPos(regionX * this.spacing + ox, regionZ * this.spacing +  
    oz);  
}
```

判定条件（isPlacementChunk）：

```
ChunkPos pos = getPotentialStructureChunk(levelSeed, chunkX, chunkZ);  
return pos.x == chunkX && pos.z == chunkZ; // 该 chunk 恰好等于网格计算的潜在  
位置才生成
```

规律：每个 34×34 区域内**最多一个**试炼密室，位于 (regionX*34+ox, regionZ*34+oz)。偏移量由区域坐标唯一决定。

1.3 种子派生：WorldgenRandom.setLargeFeatureWithSalt()

```
public void setLargeFeatureWithSalt(long worldSeed, int regionX, int  
regionZ, int salt) {  
    long m = regionX * 341873128712L + regionZ * 132897987541L + worldSeed +  
    salt;  
    this.setSeed(m);  
}
```

LegacyRandomSource.setSeed(m) = Java java.util.Random 兼容 LCG: $seed = (m \wedge 0x5DEECE66DL) \& ((1L \ll 48) - 1)$ 。每次 $nextInt(n) = next(31) \% n$ ($next(bits)$ 用经典 LCG 递推 $seed = (seed * 0x5DEECE66DL + 0xBL) \& ((1L \ll 48) - 1)$ ，取高 31/32 位)。

1.4 偏移类型：RandomSpreadType

```
public int evaluate(RandomSource random, int n) {
    return switch (this) {
        case LINEAR    -> random.nextInt(n);                // 1 次
nextInt
        case TRIANGULAR -> (random.nextInt(n) + random.nextInt(n)) / 2; // 2
次 nextInt
    };
}
```

trial_chambers 使用默认 linear，即 2 次 nextInt(22) (x 偏移一次、z 偏移一次，顺序固定)。

2. 随机数依赖（种子派生方式 + 调用顺序）

试炼密室使用 3 个相互独立的随机流。模拟/查找器必须严格区分。

2.1 随机流 A —— 区域放置偏移（定位密室所在 chunk）

- 输入：worldSeed + 区域坐标 + 盐值。
- 公式：见 §1.3，setLargeFeatureWithSalt(worldSeed, regionX, regionZ, 94251327) $\rightarrow 2 \times nextInt(22)$ 。
- 确定性、无需实际生成，只依赖世界种子和区域坐标。

2.2 随机流 B —— 结构内部 Jigsaw 拼接随机

Structure\$GenerationContext.makeRandom()：

```
private static WorldgenRandom makeRandom(long seed, ChunkPos chunkPos) {
    WorldgenRandom random = new WorldgenRandom(new LegacyRandomSource(0L));
    random.setLargeFeatureSeed(seed, chunkPos.x, chunkPos.z);
    return random;
}
```

```
public void setLargeFeatureSeed(long worldSeed, int x, int z) {
    this.setSeed(worldSeed);
    long m = this.nextLong(); // 由 worldSeed 播种的 LCG 第 1 个 long
    long n = this.nextLong(); // 第 2 个 long
}
```

```

    long o = x * m ^ z * n ^ worldSeed;
    this.setSeed(o); // 重新播种
}

```

- 输入：worldSeed + 结构起始 chunk 坐标 (= §1 计算出的潜在结构 chunk)。
- 这是一个位置型种子：同一 chunk 的结构内部随机完全一致，与之前是否消费无关。

JigsawStructure 中 B 流的调用顺序（对查找器模拟最关键）：

```

// net.minecraft.world.level.levelgen.structure.structures.JigsawStructure
public Optional<GenerationStub> findGenerationPoint(GenerationContext ctx) {
    ChunkPos chunkPos = ctx.chunkPos();
    int y = this.startHeight.sample(ctx.random(), new
WorldGenerationContext(...)); // ① nextInt(21) - 40 (uniform [-40,-20])
    BlockPos startPos = new BlockPos(chunkPos.getMinBlockX(), y,
chunkPos.getMinBlockZ());
    return JigsawPlacement.addPieces(ctx, this.startPool, ..., startPos, ...,
    PoolAliasLookup.create(this.poolAliases, startPos, ctx.seed()),
// ② 见随机流 C（不消耗 B）
    ...);
}

```

JigsawPlacement.addPieces 内 B 流消费顺序：

```

Rotation rotation = Rotation.getRandom(random); // ③ nextInt(4)
StructurePoolElement startEl = startPool.getRandomTemplate(random); // ④
nextInt(池模板总数)

```

之后进入递归（见 §3），大量消费 B 流（每次模板选择/旋转/拼接都要 nextInt 与 Fisher-Yates shuffle）。

2.3 随机流 C —— 池别名随机（决定刷怪笼内容/怪物类型）

```

// net.minecraft.world.level.levelgen.structure.pools.alias.PoolAliasLookup
static PoolAliasLookup create(List<PoolAliasBinding> bindings, BlockPos
startPos, long worldSeed) {
    RandomSource random =
RandomSource.create(worldSeed).forkPositional().at(startPos); // Xoroshiro
位置随机
    ...
    bindings.forEach(binding -> binding.forEachResolved(random, map::put));
}

```

- 输入：worldSeed + 结构起点 blockPos (chunkMinX, y, chunkMinZ)。
- 算法：Xoroshiro128++，RandomSource.create(worldSeed) 播种，forkPositional() 得到位置随机工厂，.at(startPos) 用 blockPos 做位置散列。

- 与 B 流完全独立，仅由 worldSeed + 起点坐标决定 → 同一位置的刷怪笼怪物类型是确定的。

C 流的调用顺序（TrialChambersStructurePools.ALIAS_BINDINGS，顺序固定）：

1. random_group（3 组，weight 各 1）→ nextInt(3) 选组，组内 2 条 direct 映射无随机 → 决定 ranged 与 slow_ranged 的怪种：
 - 组 0：ranged→skeleton、slow_ranged→skeleton
 - 组 1：ranged→stray、slow_ranged→stray
 - 组 2：ranged→poison_skeleton (Bogged)、slow_ranged→poison_skeleton
2. random contents/melee（3 目标）→ nextInt(3) → zombie / husk / spider
3. random contents/small_melee（4 目标）→ nextInt(4) → slime / cave_spider / silverfish / baby_zombie

WeightedList.getRandom = nextInt(总权重)，等权重时即均匀选取。

2.4 三个随机流与"查找器"的关系

流	用途	输入	决定
A	网格定位	worldSeed, regionX, regionZ, salt=94251327	密室在哪个 chunk
B	Jigsaw 拼接	worldSeed, 起始 chunk	房间组合、走廊走向、具体刷怪笼位置
C	怪物类型	worldSeed, 起点 blockPos	每个刷怪笼的 mob 种类

3. 内部结构块拼接逻辑（数据驱动 Jigsaw）

3.1 结构配置：

data/minecraft/worldgen/structure/trial_chambers.json

关键字段：

```
{
  "type": "minecraft:jigsaw",
  "start_pool": "minecraft:trial_chambers/chamber/end",
  "size": 20,
  "start_height": { "type": "minecraft:uniform", "max_inclusive":
{"absolute": -20}, "min_inclusive": {"absolute": -40} },
  "use_expansion_hack": false,
  "max_distance_from_center": 116,
  "dimension_padding": 10,
  "step": "underground_structures",
```

```
"pool_aliases": [ ... 见 §2.3 ... ]
}
```

3.2 起始块 (START)

`TrialChambersStructurePools.bootstrap` :

```
bootstrapContext.register(START /* trial_chambers/chamber/end */, new
StructureTemplatePool(
    holder, List.of(
        Pair.of(single("trial_chambers/corridor/end_1", processor), 1),
        Pair.of(single("trial_chambers/corridor/end_2", processor), 1)
    ), Projection.RIGID));
```

→ 起点模板 50%: `corridor/end_1` , 50%: `corridor/end_2` (B 流 `nextInt(2)`)。

`corridor/end_1` / `end_2` 的 jigsaw (`pool` 字段):

- `end_1` : `chambers/end` ×1、`corridor` ×1、`reward/ominous_vault` ×1
- `end_2` : `chambers/end` ×1、`corridor` ×1、`reward/all` ×1、`reward/ominous_vault` ×1

3.3 房间池与权重 (`TrialChambersStructurePools` + JSON)

`trial_chambers/chambers/end` (第一个房间): `chamber_1` / `assembly` / `eruption` / `slanted` 各 weight 1。

`trial_chambers/hallway` (走廊连接, 权重关键):

模板	权重
<code>chamber/chamber_1</code>	150
<code>chamber/chamber_2</code>	150
<code>chamber/chamber_4</code>	150
<code>chamber/chamber_8</code>	150
<code>chamber/assembly</code>	150
<code>chamber/eruption</code>	150
<code>chamber/slanted</code>	150
<code>chamber/pedestal</code>	150
<code>hallway/rubble_chamber</code>	10
其余走廊连接件 (<code>corridor_connector</code> 、 <code>upper/lower_hallway_connector</code> 、 <code>straight</code> 、 <code>corner</code> 、 <code>encounter_1..5</code> 等)	各 1

→ 主房间几乎总是 8 种 chamber 之一（每种约 1/8 概率，取决于拼接时是否能放置）。

各 chamber 模板的子池（chamber_1 示例，jigsaw pool 字段统计）：

目标池	数量	说明
trial_chambers/dispensers/chamber	15	发射器（可带箭/水，weight 1:1:1:1:1 → 有空、chamber、wall、floor）
trial_chambers/chamber/addon	5	附加件池（10 元素各 weight 1，含 c1_breeze → 1 只旋风人刷怪笼）
trial_chambers/chamber/entrance_cap	4	入口封帽（jigsaw→empty 终止）
trial_chambers/decor/disposal	4	装饰
trial_chambers/reward/ominous_vault	1	宝库（vault 块）

3.4 递归拼接算法（JigsawPlacement\$Placer.tryPlacingChildren）

伪代码要点：

```
对当前 piece 的每个 jigsaw 块（顺序：getShuffledJigsawBlocks 洗牌后）：
    targetPool = poolAliasLookup.lookup(该块的 pool)
    模板候选 = (深度 < maxDepth) ? 主池.getShuffledTemplates(random) : []
    模板候选 += fallback 池.getShuffledTemplates(random)
    for 每个候选模板（按洗牌顺序）：
        if 候选 == EmptyPoolElement: break
        for 每个旋转（Rotation.getShuffled）：
            if JigsawBlock.canAttach(当前块, 候选模板的某 jigsaw 块):
                计算连接位移（块坐标差 + 朝向 + 刚性投影对齐）
                检查 AABB 重叠（Shapes.joinIsEmpty, deflate 0.25）→ 不重叠才
放置
                放置新 PoolElementStructurePiece, 加入 pieces 列表
                若深度+1 <= maxDepth, 入队递归
                跳到下一个 jigsaw 块（同块只成功连接一次）
```

- **maxDepth = 20**；半径限制 116 格（max_distance_from_center）。
- **JigsawBlock.canAttach**：两侧 jigsaw 朝向相对，且 对方块的 name == 本块的 target。

4. 试炼刷怪笼（Trial Spawner）的生成位置与数量

4.1 关键架构变化

1.21.11 中刷怪笼不是直接内嵌在 chamber 模板里，而是通过「jigsaw → spawner 池 → connector → mob 模板」的连接链逐层放置。每个 mob 模板恰好含 1 个 trial_spawner

块。

全 191 个 trial_chambers 模板中，含 trial_spawner 块的只有 14 个：

模板	刷怪笼数
spawner/ranged/{skeleton,stray,poisson_skeleton}	各 1
spawner/slow_ranged/{skeleton,stray,poisson_skeleton}	各 1
spawner/melee/{zombie,husk,spider}	各 1
spawner/small_melee/{slime,cave_spider,silverfish,baby_zombie}	各 1
spawner/breeze/breeze	1

4.2 连接链（一条链 = 1 个刷怪笼）

```
chamber 模板的 jigsaw 块 (pool = "trial_chambers/spawner/<类别>")
  → 池 trial_chambers/spawner/<类别>          (ranged / slow_ranged / melee /
small_melee / breeze / all)
  → 模板 spawner/connectors/<类别>.nbt          (1 个 jigsaw, pool =
"trial_chambers/spawner/contents/<类别>")
  → 池 trial_chambers/spawner/contents/<类别> (经 PoolAliasLookup 别名解析，见
§2.3)
  → 池 trial_chambers/spawner/<类别>/<怪种>      (如 spawner/ranged/skeleton)
  → 模板 spawner/<类别>/<怪种>.nbt              ← 内含 1 个 trial_spawner (位于模
板内 [1,1,1] 相对坐标)
```

怪物种类决定：contents/<类别> 是池别名（不是真实池）。PoolAliasLookup 用随机流 C 把 contents/ranged、contents/slow_ranged 解析到同一种怪（skeleton/stray/bogged），contents/melee 解析到 zombie/husk/spider，contents/small_melee 解析到 slime/cave_spider/silverfish/baby_zombie。

4.3 各 chamber 模板可产生的刷怪笼数量（模板内 spawner 连接点）

chamber 模板	直接 spawner 连接	附加上限
chamber_1	0	5× chamber/addon（其中 c1_breeze 概率 1/10 → 1 旋风人；full_stacked_walkway* 概率 1/10 → 1 ranged）
chamber_2	1× spawner/ranged	9× chamber/addon
chamber_4	2× spawner/small_melee	+ 装饰/奖励池
chamber_8	4× spawner/slow_ranged	+ 装饰/奖励池

chamber 模板	直接 spawner 连接	附加上限
assembly	1× spawner/breeze + 1× spawner/ranged	24× chamber/assembly 子池（子模板 left/right_staircase_3 各可再带 1× spawner/all）
eruption	0（子模块决定）	子模板（eruption 池）含 spawner 连接
slanted	0（子模块决定）	slanted/center → 1×breeze； quadrant_1 → 1×melee； ramp_1/2 → 1×melee； ramp_3 → 1×ranged
pedestal	0（子模块决定）	子模板含 spawner 连接
走廊附加	—	corridor/addon/open_walkway → 1× spawner/all； walled_walkway → 1× spawner/all； open_walkway_upper → 1× spawner/ranged；中庭装饰部分亦有

注意：chamber_2/4/8 的命名**不再等于**刷怪笼数量（旧 1.21.1 是 1/2/4/8）。1.21.11 中它们是直接连接数：chamber_2=1、chamber_4=2、chamber_8=4；而 chamber_1 本身 0 直接，靠 addon 随机补。

4.4 刷怪笼块实体配置（normal / ominous）

每个 trial_spawner 块实体的 NBT：

```
normal_config = minecraft:trial_chamber/<类别>/<怪种>/normal
ominous_config = minecraft:trial_chamber/<类别>/<怪种>/ominous
```

运行时由 TrialSpawnerBlockEntity 根据是否激活不祥试炼选择 normal/ominous 配置（TrialSpawnerConfigs 数据注册于 data/minecraft/trial_spawner/trial_chamber/**/normal.json 与 ominous.json）。

配置关键字段（TrialSpawnerConfig）：

字段	含义	例（melee/zombie normal）
simultaneous_mobs	同时在场怪物基数	3.0
simultaneous_mobs_added_per_player	每多一名玩家增加	0.5
ticks_between_spawn	两次生成间隔（tick）	20
total_mobs	一波总生成数（仅部分配置）	breeze normal = 2.0

字段	含义	例 (melee/zombie normal)
total_mobs_added_per_player	每玩家增加总生成数	breeze normal = 1.0
spawn_potentials	生成实体池	zombie; ominous 时带装备表 equipment/trial_chamber_mele
loot_tables_to_eject	击杀后掉落 (ominous)	spawners/ominous/trial_chamb + consumables (w7)

4.5 刷怪笼位置

位置 = 对应 mob 模板 (如 spawner/ranged/skeleton.nbt) 在整座密室中的**绝对坐标**。每个 mob 模板内刷怪笼位于模板相对坐标 [1,1,1]，实际位置 = 模板放置原点 + 旋转/镜像变换。要精确求位置，必须完整模拟 B 流 Jigsaw 拼接 (§3.4)。

5. 模拟计算 / 查找器实现指引

5.1 定位密室（只依赖随机流 A）—— 精确可算

```
SPACING, SEPARATION, SALT = 34, 12, 94251327
```

```
def legacy_random_seed(seed): # java.util.Random / LegacyRandomSource
    return (seed ^ 0x5DEECE66D) & ((1 << 48) - 1)
```

```
def next_long(rng): # LegacyRandomSource.nextLong() = (next(32)
    <<32) + next(32)
    rng = (rng * 0x5DEECE66D + 0xB) & ((1 << 48) - 1); hi = rng >> 16
    rng = (rng * 0x5DEECE66D + 0xB) & ((1 << 48) - 1); lo = rng >> 16
    return (hi << 32) + lo, rng
```

```
def next_int(rng, n):
    rng = (rng * 0x5DEECE66D + 0xB) & ((1 << 48) - 1)
    return (rng >> 17) % n, rng # 注意: java Random nextInt(n) 有拒绝采
样, n 非 2 的幂时需完整实现
```

```
def set_large_feature_with_salt(world_seed, rx, rz, salt):
    m = rx * 341873128712 + rz * 132897987541 + world_seed + salt
    return legacy_random_seed(m)
```

```
def potential_chunk(world_seed, rx, rz):
    rng = set_large_feature_with_salt(world_seed, rx, rz, SALT)
    ox, rng = next_int(rng, SPACING - SEPARATION) # nextInt(22)
    oz, rng = next_int(rng, SPACING - SEPARATION)
    return rx * SPACING + ox, rz * SPACING + oz
```

遍历所有 `regionX`, `regionZ` 即可枚举整个世界所有试炼密室的起始 chunk。

注意： `java.util.Random.nextInt(n)` 对非 2 的幂 `n` 有**拒绝采样** (`(bits - val + (n - 1)) < 0` 时重试)，22 不是 2 的幂，必须精确复现 Java 的 `nextInt` 实现，否则偏移错位。

5.2 判断密室是否真正生成（需生物群系检查）

`Structure.findValidGenerationPoint` 会验证起点 `biome` $\in$ `#has_structure/trial_chambers (getNoiseBiome(QuartPos...))`。密室实际生成还受群系条件约束；`StructureCheck` 的 `canCreateStructure` 在放置前会调用该检查。

5.3 刷怪笼怪物类型（随机流 C）—— 精确可算

```
# Xoroshiro128++ 位置随机:
RandomSource.create(world_seed).forkPositional().at(startBlockPos)
# startBlockPos = (chunkX*16, y, chunkZ*16), y = nextIntB(21) - 40 (B 流第@次调用)
alias_rng = xoroshiro_at(world_seed, startBlockPos)
group = alias_rng.nextInt(3)           # 决定 ranged/slow_ranged 怪种
melee = alias_rng.nextInt(3)           # zombie / husk / spider
small = alias_rng.nextInt(4)           # slime / cave_spider / silverfish /
baby_zombie
```

(`forkPositional().at(pos)` 的精确散列 = `fromHash0f` 用种子与 `blockPos` 混合，需实现 Xoroshiro 的位置工厂 `at(x,y,z)`。)

5.4 精确刷怪笼数量/位置（随机流 B）—— 需完整 Jigsaw 模拟

需要：

1. 反编译实现的 `JigsawPlacement$Placer` (§3.4)；
2. 全部池 JSON (47 个，已从 jar 提取) 与模板 NBT (191 个，含每个 jigsaw 块的 `pool/name/target` 与每个 `trial_spawner` 块坐标)；
3. 严格按 B 流调用顺序消费 `WorldGenRandom` (初始 `setLargeFeatureSeed(worldSeed, startChunkX, startChunkZ)`)。

若只需要「哪个 region 有密室 + 大约多少刷怪笼」，可用 §5.1 + §4.3 的权重表做概率模拟；若需要逐格精确，则必须完整跑 Jigsaw 拼接。

6. 附录：关键反编译文件清单

文件	作用
RandomSpreadStructurePlacement.java	34×34 网格放置算法、种子派生
WorldgenRandom.java	setLargeFeatureSeed / setLargeFeatureWithSalt
RandomSpreadType.java	linear/triangular 偏移
Structure\$GenerationContext.java	结构内部随机 makeRandom
JigsawStructure.java	起始高度采样、调用 JigsawPlacement
JigsawPlacement.java (含 \$Placer、\$PieceState)	递归拼接、重叠检测、随机消费顺序
StructureTemplatePool.java	加权模板选择 getRandomTemplate / getShuffledTemplates
PoolAliasLookup.java / RandomPoolAlias.java / RandomGroupPoolAlias.java	池别名（怪物类型）随机解析
TrialChambersStructurePools.java	全部试炼密室池注册与 ALIAS_BINDINGS
TrialSpawnerConfigs.java / TrialSpawnerConfig.java	刷怪笼 normal/ominous 配置数值
191 个 data/minecraft/structure/trial_chambers/**/*.nbt	模板（含 jigsaw 连接点与 trial_spawner 坐标）
47 个 worldgen/template_pool/trial_chambers/**/*.json	池定义与权重
worldgen/structure_set/trial_chambers.json + worldgen/structure/trial_chambers.json	放置网格 + Jigsaw 结构配置